CS101 • SOFTWARE ENGINEERING FUNDAMENTALS

Developer Core Toolkit: Git & REST APIs

Mastering distributed version control and modern web application interfaces.

PART 1
Git Version Control

PART 2
RESTful Web Services

SESSION AGENDA

Module Breakdown & Learning Goals

MODULE 01

Git & Source Control

Core Model: Snapshots vs Delta tracking

4 Stages: Working, Staging, Local, Remote

Workflows: Branching, merging, conflicts

MODULE 02

REST API Architecture

HTTP Verbs: GET, POST, PUT, DELETE

Payloads: JSON formatting and headers

Status Codes: 2xx, 4xx, 5xx standards

VERSION CONTROL

Why Developers Depend on Git

Time Machine

Instant rollback to any historical commit snapshot.

Isolated Branching

Safely develop features without breaking main code.

Async Collaboration

Seamless parallel development across globally distributed teams.

GIT ARCHITECTURE

The Four Stages of Code

STEP 01

Working Directory

Unmodified and untracked file edits.

STEP 02

Staging Area

Prepared changes ready for commit.

STEP 03

Local Repo

Committed historical snapshots on machine.

STEP 04

Remote Repo

Cloud server repository like GitHub.

CHEATSHEET

Essential Everyday Commands

COMMAND	TARGET ACTION	TYPICAL WORKFLOW STEP
`git init`	Initialize new local Git repository	Project setup phase
`git add .`	Stage modified files for commit	Pre-commit packaging
`git commit -m "msg"`	Save staged snapshot with message	Checkpoint creation
`git push origin main`	Upload commits to remote server	Publishing updates
`git pull`	Fetch and merge remote changes	Syncing local copy

COLLABORATION PATTERNS

Branching Strategies

Feature Isolation

Command: `git checkout -b feat/login`

Keep experimental work away from main line.

Allows multiple team members to work simultaneously.

Integration & Merge

Command: `git merge feat/login`

Code reviews validate security and quality.

Automated CI/CD runs before code enters production.

PART TWO

REST API Fundamentals

Connecting client applications with backend server resources across HTTP.

WEB INTERFACES

What Makes an API "RESTful"?

Client-Server Separation
User interfaces and database logic evolve independently.

Resource-Based Endpoints
URIs represent data entities (e.g., `/api/users`).

Stateless Requests
Every request contains full authentication credentials.

Uniform Interface
Standardized HTTP methods manipulate server data.

HTTP METHODS

Mapping Verbs to CRUD Actions

GET

READ DATA

Retrieves resources without modifying server state.

POST

CREATE RESOURCE

Submits new data payload to target endpoint.

PUT

UPDATE / REPLACE

Replaces existing target resource with payload.

DELETE

REMOVE

Removes target resource record permanently.

PAYLOAD FORMAT

Anatomy of a REST Request

```
// HTTP POST Request Example

POST /api/v1/students HTTP/1.1

Host: api.university.edu
Content-Type: application/json
Authorization: Bearer token_xyz

{
  "name": "Alex Rivera",
  "major": "Computer Science"
}
```

Endpoint URL & Path
Identifies server domain and resource target.

Request Headers
Passes auth tokens and expected metadata format.

JSON Body Payload
Contains structured key-value data sent to server.

SERVER RESPONSES

Essential HTTP Status Codes

2xx

Success

200: OK
201: Created

4xx

Client Error

400: Bad Request
401: Unauthorized
404: Not Found

5xx

Server Error

500: Internal Error
503: Service Unavailable